UFRGS
UNIVERSIDADE FEDERAL DO RIO GRANDE DO SUL

Curso de Especialização

Inteligência Artificial para Engenheiros

Análise de Dados e Engenharia de Características

AULA 04B

Lidando e Revisando Problemas Comuns dos Dados: Dados e Classes Desbalanceadas

Professor Dr. Alexandre Balbinot

2026

IA PARA ENGENHEIROS
UFRGS

Universidade Federal do Rio Grande do Sul
Escola de Engenharia
Curso de Especialização Inteligência Artificial para Engenheiros
Laboratório de Sistemas Eletrônicos Inteligentes (LASEI)

Análise de Dados e Engenharia de Características

Aula 04B

Lidando e Revisando Problemas Comuns dos Dados: Dados e Classes Desbalanceadas

Prof. Dr. Alexandre Balbinot

Contatos

Prof. Dr. Alexandre ***Balbinot***

Curso de Especialização Inteligência Artificial para Engenheiros

Escola de Engenharia – UFRGS

Prédio do Instituto Eletrotécnico

Departamento de Engenharia Elétrica - DELET

Laboratório de Sistemas Eletrônicos Inteligentes (LASEI)

Sala 206D – (51) 3308-**4442**

alexandre.balbinot@ufrgs.br

Direitos Autorais

Este material possui Direitos Autorais e Direitos de Imagem e não pode ser duplicado, reproduzido, no todo ou em parte, sob quaisquer formas ou por quaisquer meios (eletrônicos, mecânico, gravação, fotocópia, distribuição na internet ou outros), sem permissão expressa do autor.

©

Termo de Responsabilidade da Própria UFRGS

Todos os materiais disponibilizados nos Ambientes Virtuais da UFRGS são exclusivamente para usos e fins didáticos, sendo vedada a sua utilização para qualquer outra finalidade, sob as penas legais.

Todos os materiais de terceiros que venham a ser utilizados ou disponibilizados devem ser referenciados, sob as penas legais, inclusive de plágio.

Tenho liberdade de expor ou não a minha imagem e voz nas atividades, o que não me desonera da participação efetiva nas mesmas.

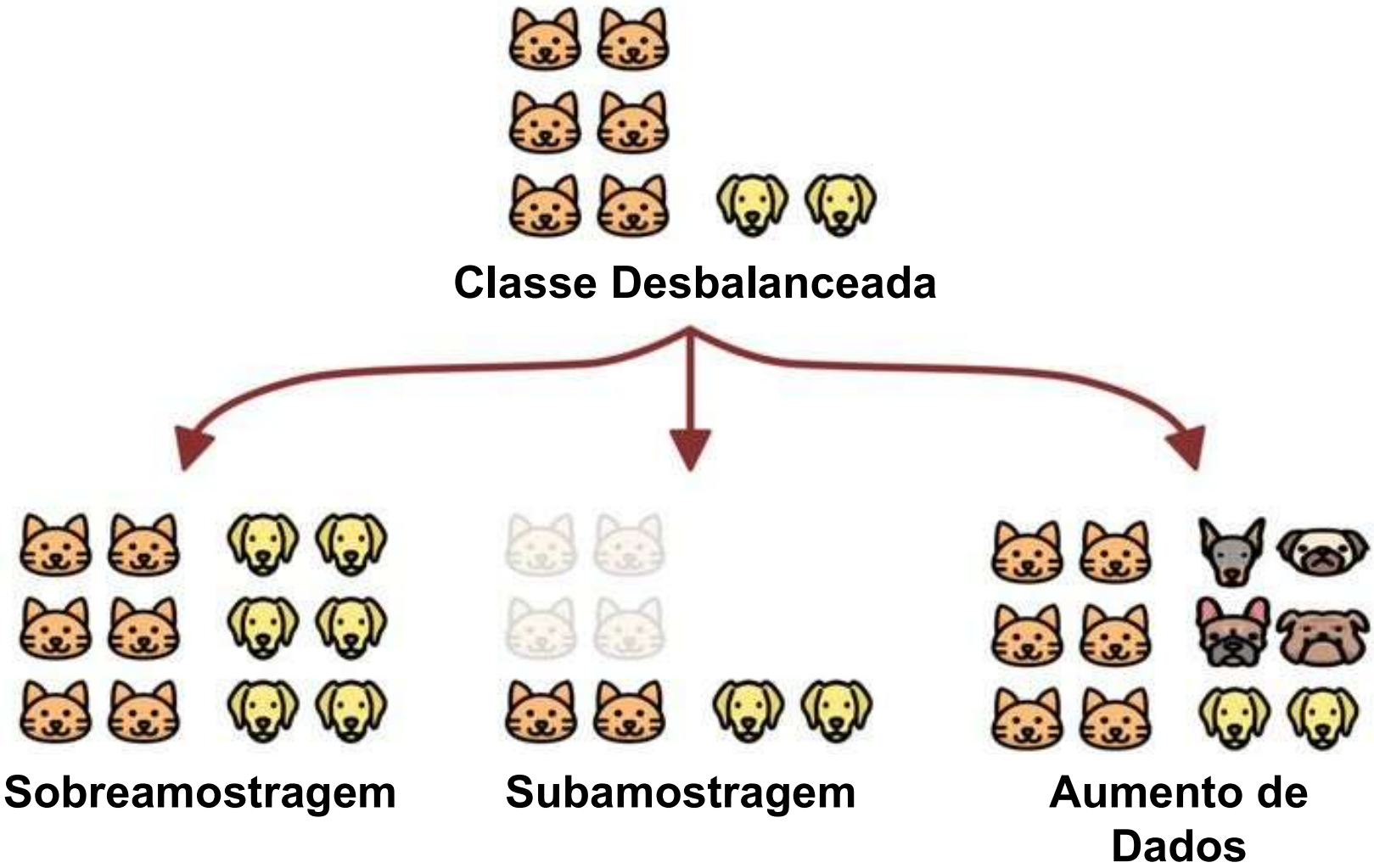
Todas as gravações de atividades síncronas devem ser previamente informadas por parte dos professores.

Os alunos somente poderão gravar, armazenar ou compartilhar atividades mediante concordância prévia do professor, sob as penas legais.

Os alunos somente poderão armazenar ou compartilhar materiais mediante concordância prévia do professor ou se permitido no material, sob as penas legais.

É proibido disponibilizar, por quaisquer meios digitais ou físicos, os dados, a imagem e a voz dos alunos, dos colegas e do professor, sem autorização específica para a finalidade pretendida.

INTRODUÇÃO


Classe Desbalanceada
Sobreamostragem
Subamostragem
Aumento de Dados

Exemplos

<u>Exemplo 01</u>: análise preliminar de um conjunto de dados

Considere um experimento aleatorizado cujo objetivo é realizar a medição da tensão mecânica em uma máquina de ensaios com quatro tipos de corpos de prova de cimentos (ver Tabela 1). Importante observar que para facilitar nossa apresentação vamos considerar um tamanho amostral de apenas 16 amostras. Analisando-se a Tabela 1 responder: (a) Qual a variável de resposta ou de medida? (b) Qual o fator controlável deste experimento? (c) Esta base de dados é balanceada?

Tabela 1 – Exemplo de um conjunto de dados com *N.. = 16*.

Tipo de Mistura	**Tensão Mecânica (lb/in^2)**			
Classe 1	3129	3000	2865	2890
Classe 2	3200	3300	2975	3150
Classe 3	2800	2900	2985	3050
Classe 4	2600	2700	2600	2765

<u>Exemplo 02</u>: análise preliminar de um conjunto de dados

Considerar o mesmo experimento aleatorizado anterior. Porém, neste caso, a coleta de dados segue a representação da Tabela 2. Importante observar que o tamanho amostral permanece o mesmo do <u>Exemplo 01</u>. Esta base de dados é balanceada?

Tabela 2 – Exemplo de um conjunto de dados com *N.. = 16*.

TIPO DE MISTURA	TENSÃO MECÂNICA (LB/IN^2)					
CLASSE 1	3129	3000	2865	2890	2895	2999
CLASSE 2	3200	3300				
CLASSE 3	2800	2900	2900	2899		
CLASSE 4	2600	2700	2600	2599		

<u>Exemplo 03</u>: verificar o arquivo *wdbc.data* e responder:

(a) Esta base é supervisionada?
(b) Qual o tamanho amostral desta base de dados?
(c) Quantas classes a mesma apresenta?
(d) É uma base de dados balanceada?

https://www.kaggle.com/datasets/uciml/breast-cancer-wisconsin-data

Search

UCI MACHINE LEARNING · UPDATED 6 YEARS AGO

2745

Breast Cancer Wisconsin (Diag
Data Set

Predict whether the cancer is benign or malignant

Data Code (2186) Discussion (49) Metadata

About Dataset

Features are computed from a digitized image of a fine needle aspirate (FNA) of
characteristics of the cell nuclei present in the image.
n the 3-dimensional space is that described in: [K. P. Bennett and O. L. Mangasa
Programming Discrimination of Two Linearly Inseparable Sets", Optimization Met

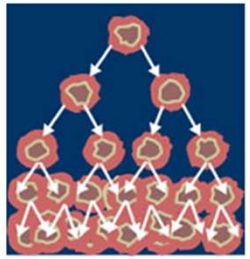
https://archive.ics.uci.edu/ml/datasets/Breast+Cancer+Wisconsin+%28Diagnostic%29

Machine Learning Repository
Center for Machine Learning and Intelligent Systems

Check out the beta version of the new UCI Machine Learning Repository we are currently testing! Contact us if you have any i

Breast Cancer Wisconsin (Diagnostic) Data Set
Download: Data Folder, Data Set Description

Abstract: Diagnostic Wisconsin Breast Cancer Database



Data Set Characteristics:	Multivariate	**Number of Instances:**	569	**Area:**	Life
Attribute Characteristics:	Real	**Number of Attributes:**	32	**Date Donated**	1995-11-01
Associated Tasks:	Classification	**Missing Values?**	No	**Number of Web Hits:**	1755676

Source:

Creators:

1. Dr. William H. Wolberg, General Surgery Dept.
University of Wisconsin, Clinical Sciences Center
Madison, WI 53792
wolberg '@' eagle.surgery.wisc.edu

2. W. Nick Street, Computer Sciences Dept.
University of Wisconsin, 1210 West Dayton St., Madison, WI 53706

Mitigando o Desbalanço dos Dados

Principais Métodos para Mitigar o Desbalanço

- determinar um peso para as classes;
- sub-amostrar;
- sobreamostrar;
- combinação de sobreamostragem e subamostragem;
- reamostragem de conjunto.

Métodos de reamostragem ajudam a equilibrar a distribuição das classes, mas nem sempre melhoram o desempenho do modelo.
Avaliar os modelos com um conjunto de validação separado e intocado e considerar outras estratégias, como a escolha de métricas de avaliação apropriadas, é crucial.

Sugiro fortemente a consulta e uso:

https://imbalanced-learn.org/stable/

Imbalanced learn

Getting Started | User Guide | API reference | Examples | Release history | About us

Search the docs ...

imbalanced-learn documentation

Date: May 17, 2022 **Version**: 0.9.1

Useful links: Binary Installers | Source Repository | Issues & Ideas | Q&A Support

Imbalanced-learn (imported as `imblearn`) is an open source, MIT-licensed library relying on scikit-learn (imported as `sklearn`) and provides tools when dealing with classification with imbalanced classes.

Getting started

Check out the getting started guides to install *imbalanced-learn*. Some extra information to get started with a new

User guide

The user guide provides in-depth information on the key concepts of imbalanced-learn with useful

Section Navigation

- Under-sampling methods
- Over-sampling methods
  - RandomOverSampler
  - SMOTE
  - SMOTENC
  - SMOTEN
  - ADASYN
  - BorderlineSMOTE
  - KMeansSMOTE
  - SVMSMOTE
- Combination of over- and under-sampling methods
- Ensemble methods
- Batch generator for Keras
- Batch generator for TensorFlow
- Miscellaneous
- Pipeline
- Metrics
- Model selection methods
- Datasets
- Utilities

RandomOverSampler

class imblearn.over_sampling.RandomOverSampler(*, *sampling_strategy*='auto', *random_state*=None, *shrinkage*=None) [source]

Class to perform random over-sampling.

Object to over-sample the minority class(es) by picking samples at random with replacement. The bootstrap can be generated in a smoothed manner.

Read more in the User Guide.

Parameters:

sampling_strategy : *float, str, dict or callable, default='auto'*

Sampling information to resample the data set.

- When `float`, it corresponds to the desired ratio of the number of samples in the minority class over the number of samples in the majority class after resampling. Therefore, the ratio is expressed as $\alpha_{os} = N_{rm}/N_M$ where N_{rm} is the number of samples in the minority class after resampling and N_M is the number of samples in the majority class.

> **Warning**
>
> `float` is only available for **binary** classification. An error is raised for multi-class classification.

Algumas Excelentes Referências

Algumas excelentes referências relacionadas a geração de dados sintéticos para a etapa de treinamento:

- Nitesh V. Chawla, Kevin W. Bowyer, Lawrence O. Hall, W. Philip Kegelmeyer. *SMOTE: Synthetic Minority Over-sampling Technique*. Journal of Artificial Intelligence Research, 16: 321-357, 2002.
- Guillaume Lematre, Fernando Nogueira, Christos K. Aridas. *Imbalanced-learn: A Python Toolbox to Tackle the Curse of Imbalanced Datasets in Machine Learning*. Journal of Machine Learning Research, 18: 1-5, 2017.

Algumas sugestões para consultar em *sites*:

https://www.kdnuggets.com/2017/06/7-techniques-handle-imbalanced-data.html
https://www.kdnuggets.com/2016/08/learning-from-imbalanced-classes.html
https://www.kdnuggets.com/2016/04/unbalanced-classes-svm-random-forests-python.html
https://www.kdnuggets.com/2017/01/tidying-data-python.html

Algoritmos mais Famosos para Reamostragem de Dados: SMOTE e ADASYN

Comparação dos Métodos de Amostragem

Classes	Número de Dados do Conjunto Original	Número de Dados Após a Amostragem		
		Sub-amostragem	Sobre-amostragem	SMOTE
1	1000	1000	5000 (repetição de dados existentes)	5000 (novos dados gerados)
2	5000	1000	5000	5000

Ler Artigos Disponibilizados – Paper01.rar

O Algoritmo SMOTE

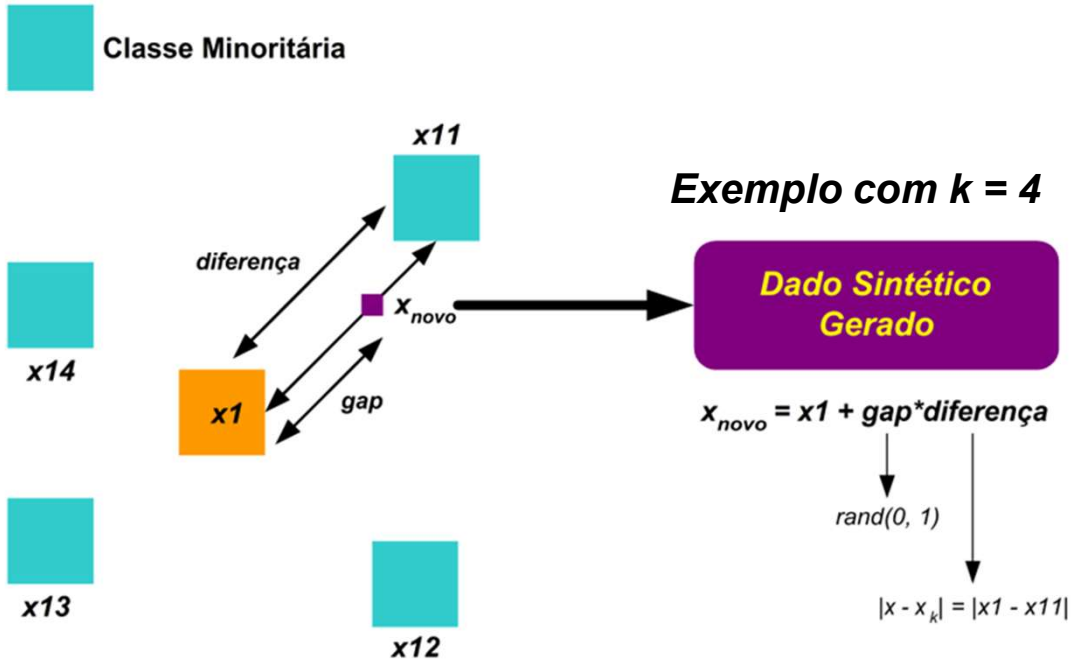
(*Synthetic Minority Over-sampling Technique*)

Passo 1:

- classe minoritária é chamada de UM (conjunto de $x \in A$);
- determinado os k-NN de x;

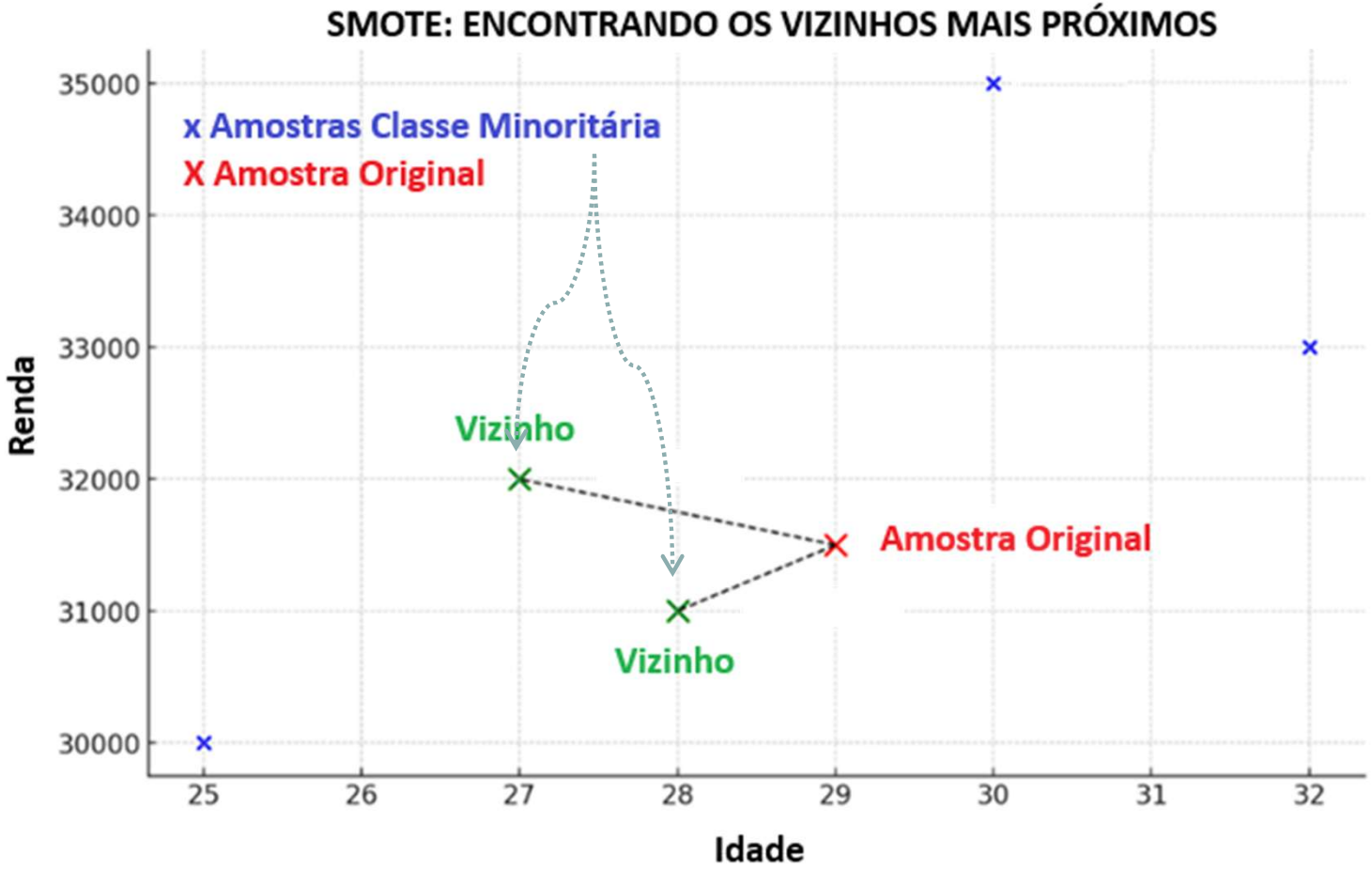
- Passo 2:
  - definir uma taxa de amostragem N de acordo com a proporção do desequilíbrio;
  - do conjunto $x \in A$, N dados são selecionados aleatoriamente dos k-NN;
  - esses dados formam o conjunto A_1;
- Passo 3:
  - para cada $x_k \in A_1$ $(k = 1, 2, 3, \ldots, N)$ o novo dado é gerado de acordo com uma equação linear:

$$x_{novo} = x + rand(0,1) * |x - x_k|$$

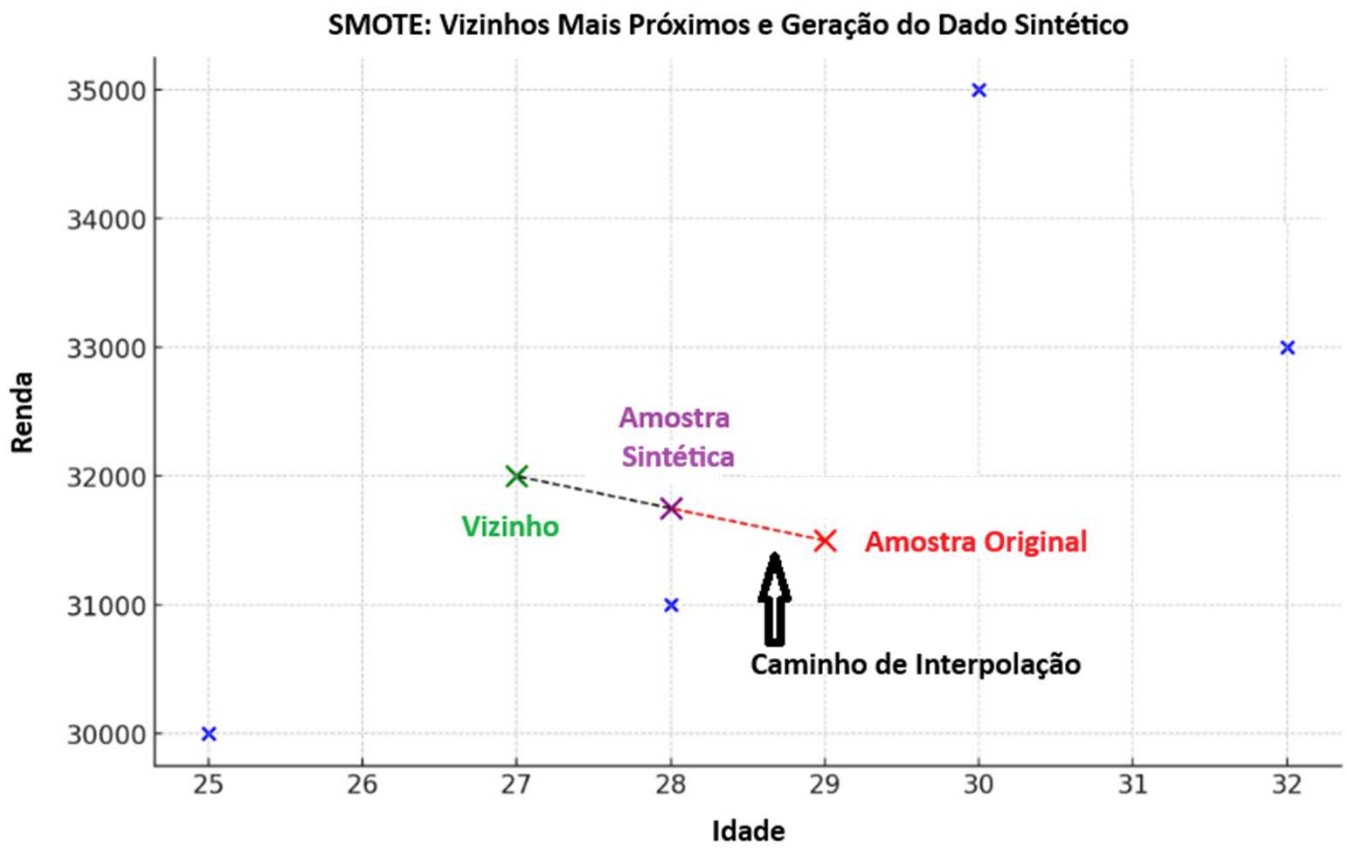


<u>Exemplo 04</u>: primeira etapa do algoritmo SMOTE

Suponha que você tenha um conjunto de dados com características que representam diferentes atributos relacionados a um conjunto de pessoas (como idade, renda, etc). Imagine que você deseja classificar algum evento raro relacionado as pessoas (Classe Minoritária) com o algoritmo SMOTE. Este algoritmo primeiro identificará algumas pessoas da Classe Minoritária que possuam características semelhantes.



Etapa 2: Geração das Amostras Sintéticas



- interpolação é realizada usando:

$$x_{sintético} = x_{original} + \lambda \times \left(x_{vizinho} - x_{original}\right)$$

<u>Exemplo 05</u>: exemplo da interpolação

Suponha dois dados da classe minoritária:
Dado 1: (Idade: 25, Renda: R$ 30.000);
Dado 2: (Idade: 30, Renda: R$ 35.000).
O SMOTE pode gerar um exemplo sintético selecionando um valor aleatório entre esses dois dados. Considerando λ = 0,5 determinar a nova amostra sintética.

$$x_{sintético} = x_{original} + \lambda \times \left(x_{vizinho} - x_{original}\right)$$

$$x_{idade} = 25 + 0{,}5 \times (30 - 25) = 25 + 2{,}5 = 27{,}5$$

$$x_{renda} = 30000 + 0{,}5 \times (35000 - 30000) = 30000 + 2500 = 32500$$

Etapa 3: Repetir até Conjunto de Dados estar Balenceado

<u>Exemplo 06</u>: exemplo de um código com o SMOTE

[1]:
```
import pandas as pd
dados = pd.read_csv('C:/Users/abalb/Desktop/creditcard.csv')
print(dados['Class'].value_counts())
```

```
Class
0    284315
1       492
Name: count, dtype: int64
```

[2]:
```
print("\nFraudes representam {:.4f}% do dataset.\n".format((dados[dados.Class == 1].shape[0] / dados.shape[0]) * 100))
```

```
Fraudes representam 0.1727% do dataset.
```

[3]:
```
from sklearn.model_selection import train_test_split
X = dados.drop('Class', axis=1)
y = dados['Class']
X_trein, X_teste, y_trein, y_teste = train_test_split(X, y, test_size=0.3, random_state=42, stratify=y)
```

[4]:
```python
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report
from sklearn.metrics import roc_auc_score, roc_curve, accuracy_score
classificador = LogisticRegression(random_state=42)
classificador.fit(X_trein, y_trein)
y_pred = classificador.predict(X_teste)
y_proba = classificador.predict_proba(X_teste)
print("Relatório de Classificação:\n", classification_report(y_teste, y_pred, digits=4))
print("Acurácia: {:.4f}\n".format(accuracy_score(y_teste, y_pred)))
print("AUC: {:.4f}\n".format(roc_auc_score(y_teste, y_pred)))
```

```
Relatório de Classificação:
               precision    recall  f1-score   support

           0     0.9994    0.9996    0.9995     85295
           1     0.7442    0.6486    0.6931       148

    accuracy                         0.9990     85443
   macro avg     0.8718    0.8241    0.8463     85443
weighted avg     0.9989    0.9990    0.9990     85443

Acurácia: 0.9990

AUC: 0.8241
```

[5]:
```
from imblearn.over_sampling import SMOTE
smote = SMOTE(sampling_strategy = 1.0, random_state=42)
X_reamostrados, y_reamostrados = smote.fit_resample(X_trein, y_trein)
print(pd.Series(y_reamostrados).value_counts())
```

```
Class
0    199020
1    199020
Name: count, dtype: int64
```

[6]:
```
classif_smote = LogisticRegression(random_state=42)
classif_smote.fit(X_reamostrados, y_reamostrados)
y_pred_smote = classif_smote.predict(X_teste)
print("Registro da Classificação (APÓS O SMOTE):")
print(classification_report(y_teste, y_pred_smote))
print("Acurácia: {:.4f}\n".format(accuracy_score(y_teste, y_pred_smote)))
print("AUC: {:.4f}\n".format(roc_auc_score(y_teste, y_pred_smote)))
```

```
Registro da Classificação (APÓS O SMOTE):
              precision    recall  f1-score   support

           0       1.00      0.98      0.99     85295
           1       0.08      0.86      0.15       148

    accuracy                           0.98     85443
   macro avg       0.54      0.92      0.57     85443
weighted avg       1.00      0.98      0.99     85443

Acurácia: 0.9834

AUC: 0.9242
```

<u>Exemplo 07</u>: outro exemplo de um código com o SMOTE

[1]:
```
import pandas as pd
dados = pd.read_csv('C:/Users/abalb/Desktop/creditcard.csv')
print(dados['Class'].value_counts())
```

```
Class
0    284315
1       492
Name: count, dtype: int64
```

[2]:
```
from sklearn.model_selection import train_test_split
X = dados.drop('Class', axis=1)
y = dados['Class']
X_trein, X_teste, y_trein, y_teste = train_test_split(X, y, test_size=0.3, random_state=42, stratify=y)
```

[3]:
```
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report
classificador = RandomForestClassifier(random_state=42)
classificador.fit(X_trein, y_trein)
y_pred = classificador.predict(X_teste)
print("Registro da Classificação (ANTES do Algoritmo SMOTE):")
print(classification_report(y_teste, y_pred))
```

```
Registro da Classificação (ANTES do Algoritmo SMOTE):
              precision    recall  f1-score   support

           0       1.00      1.00      1.00     85295
           1       0.96      0.76      0.85       148

    accuracy                           1.00     85443
   macro avg       0.98      0.88      0.92     85443
weighted avg       1.00      1.00      1.00     85443
```

[4]:
```python
from imblearn.over_sampling import SMOTE
smote = SMOTE(sampling_strategy = 1.0, random_state=42)
X_reamostrados, y_reamostrados = smote.fit_resample(X_trein, y_trein)
print(pd.Series(y_reamostrados).value_counts())
```

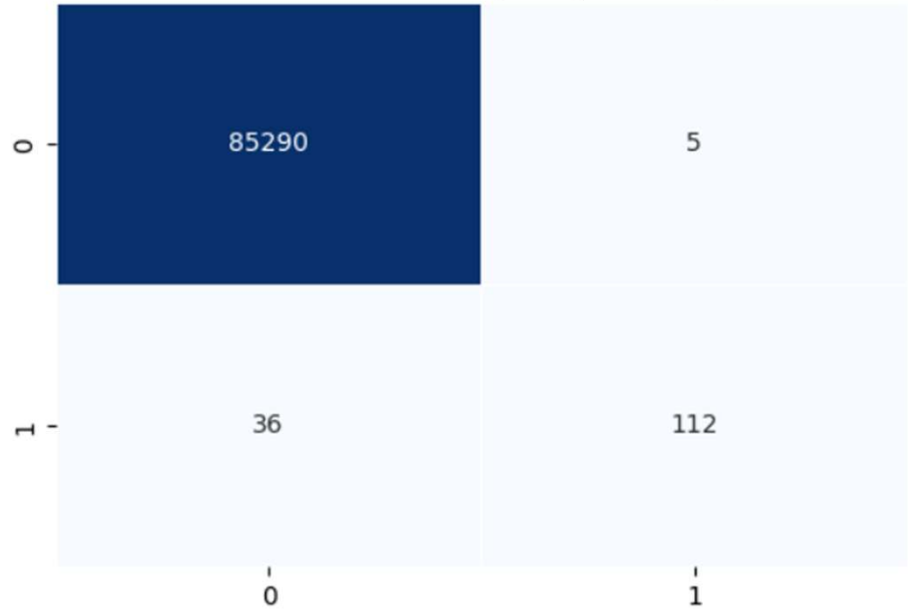
```
Class
0    199020
1    199020
Name: count, dtype: int64
```

[5]:
```python
classif_smote = RandomForestClassifier(random_state=42)
classif_smote.fit(X_reamostrados, y_reamostrados)
y_pred_smote = classif_smote.predict(X_teste)
print("Registro da Classificação (APÓS O SMOTE):")
print(classification_report(y_teste, y_pred_smote))
```

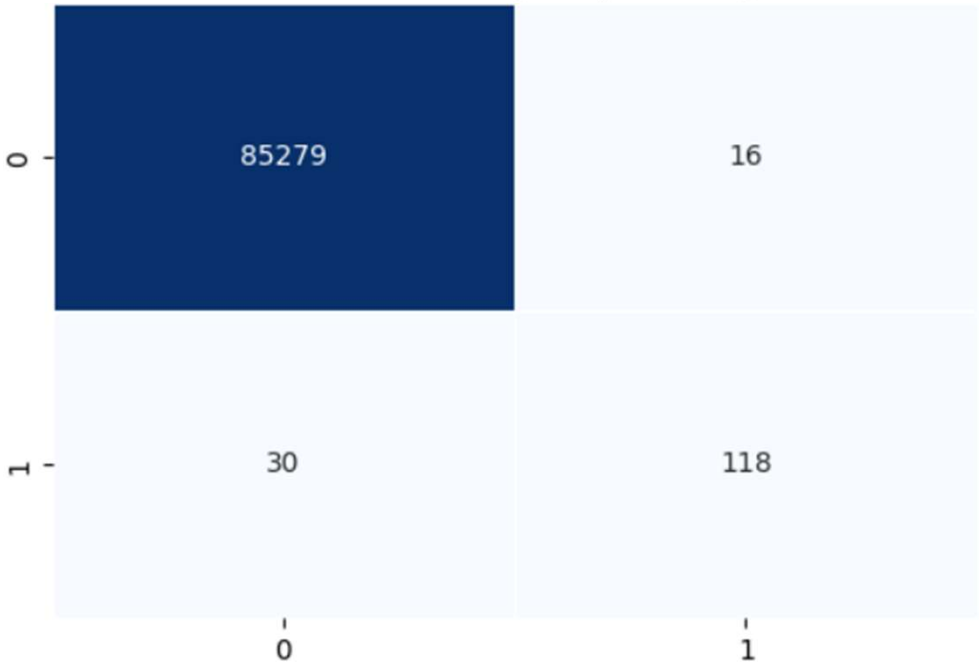
```
Registro da Classificação (APÓS O SMOTE):
              precision    recall  f1-score   support

           0       1.00      1.00      1.00     85295
           1       0.88      0.80      0.84       148

    accuracy                           1.00     85443
   macro avg       0.94      0.90      0.92     85443
weighted avg       1.00      1.00      1.00     85443
```

[6]:
```
from sklearn.metrics import confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt
matriz_confusao_ANTES = confusion_matrix(y_teste, y_pred)
plt.figure(figsize=(6, 4))
sns.heatmap(matriz_confusao_ANTES, annot=True, fmt='d', cmap='Blues', cbar=False, linewidths=0.5)
plt.title("Matriz de Confusão (ANTES da aplicação do Algoritmo SMOTE)")
plt.show()
matriz_confusao_APOS = confusion_matrix(y_teste, y_pred_smote)
plt.figure(figsize=(6, 4))
sns.heatmap(matriz_confusao_APOS, annot=True, fmt='d', cmap='Blues', cbar=False, linewidths=0.5)
plt.title("Matriz de Confusão (APÓS a aplicação do algoritmo SMOTE)")
plt.show()
```

Matriz de Confusão (ANTES da aplicação do Algoritmo SMOTE)

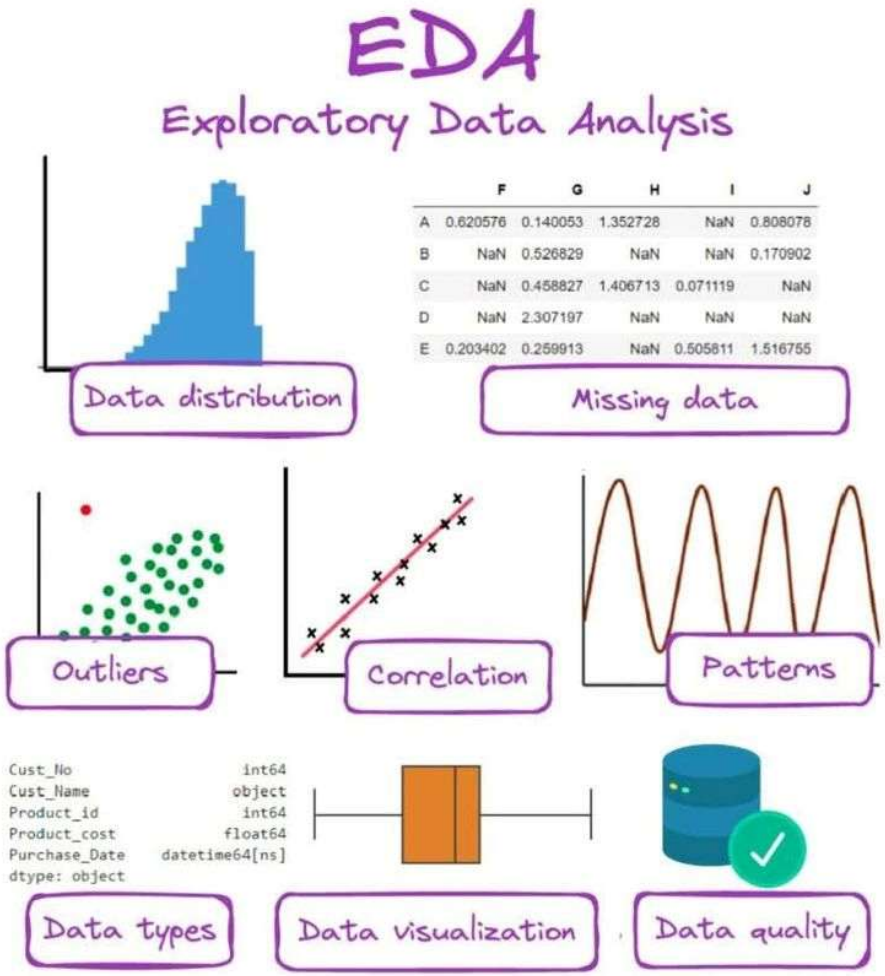


Matriz de Confusão (APÓS a aplicação do algoritmo SMOTE)



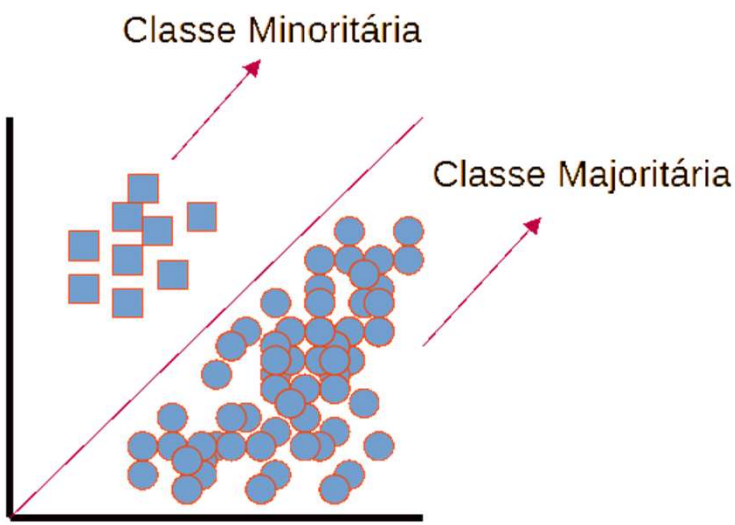
O Algoritmo ADASYN

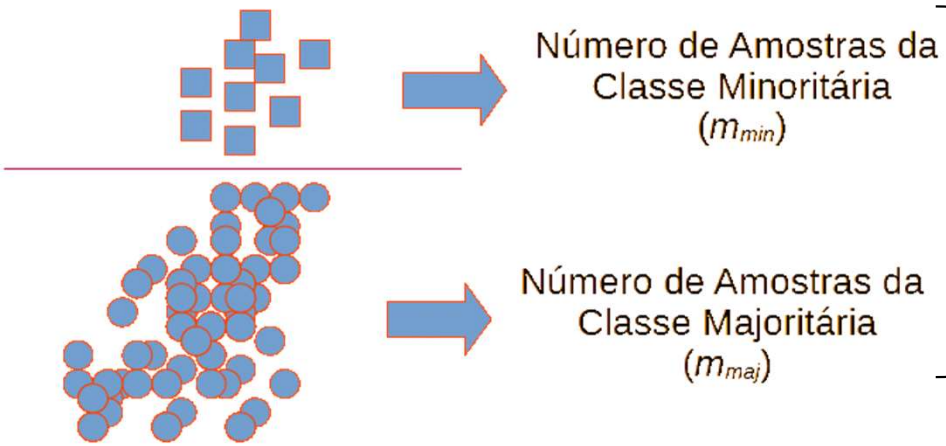
(*Adaptive Synthetic Sampling*)



	F	G	H	I	J
A	0.620576	0.140053	1.352728	NaN	0.808078
B	NaN	0.526829	NaN	NaN	0.170902
C	NaN	0.458827	1.406713	0.071119	NaN
D	NaN	2.307197	NaN	NaN	NaN
E	0.203402	0.259913	NaN	0.505811	1.516755

Etapa 1: Verificar o Grau de Desbalanço





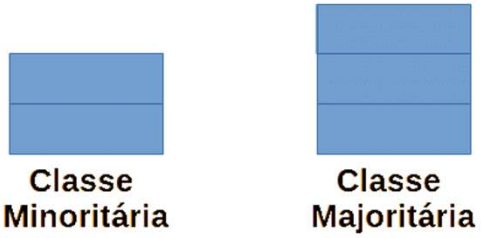
$$d = \frac{m_{min}}{m_{maj}}; d \in (0,1]$$

Etapa 2: Cálculo do Número de Dados Sintéticos

$$G = \left(m_{maj} - m_{min}\right) \times \beta$$

$\beta = 1 \rightarrow$

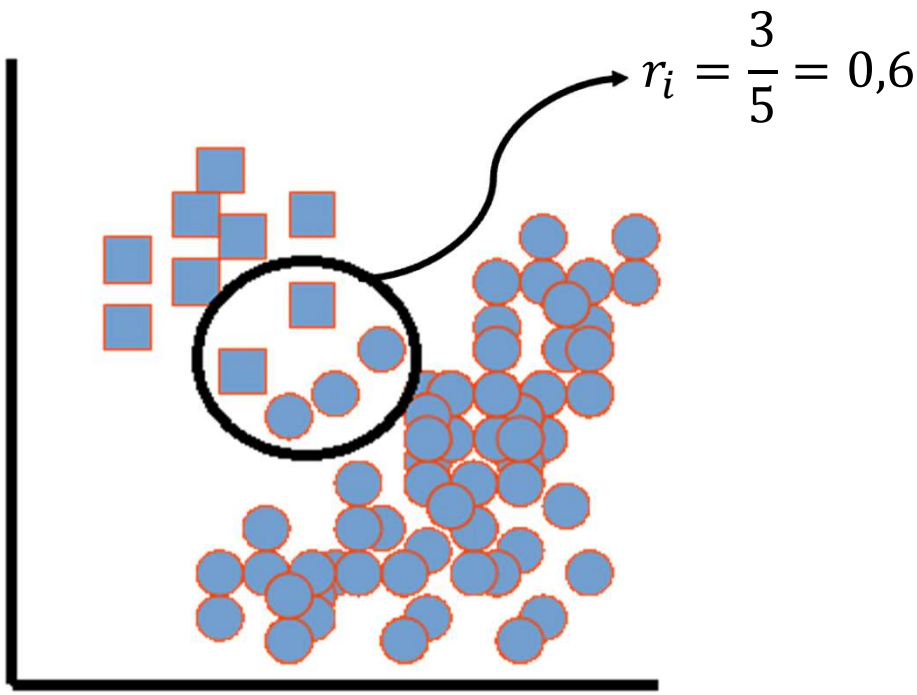
$\beta < 1 \rightarrow$



$\beta > 1 \rightarrow$

Etapa 3: Cálculo do Peso da Amostra

$$r_i = \frac{N_i}{K}, i = 1, \dots, m_s$$



Etapa 4: Normalizar o Peso e Calcular o Número de Dados Sintéticos

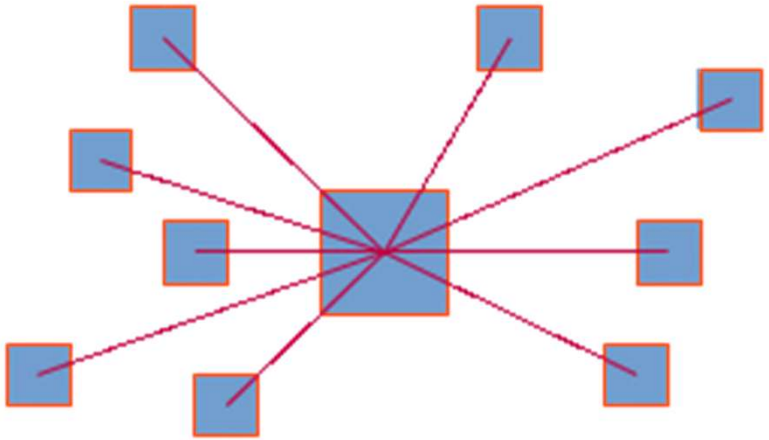
$$\widehat{r_i} = \frac{r_i}{\sum_{i=1}^{m_{min}} r_i}$$

$$g_i = \widehat{r_i} \times G$$

Considere que $r_i = \frac{3}{5} = 0{,}6$ **e** $\sum_{i=1}^{m_{min}} r_i = 20$

Logo: $\widehat{r_i} = \frac{r_i}{\sum_{i=1}^{m_{min}} r_i} = \frac{0{,}6}{20} = 0{,}03$

$$g_i = \widehat{r_i} \times G = 0{,}03(400 - 100) = 9$$

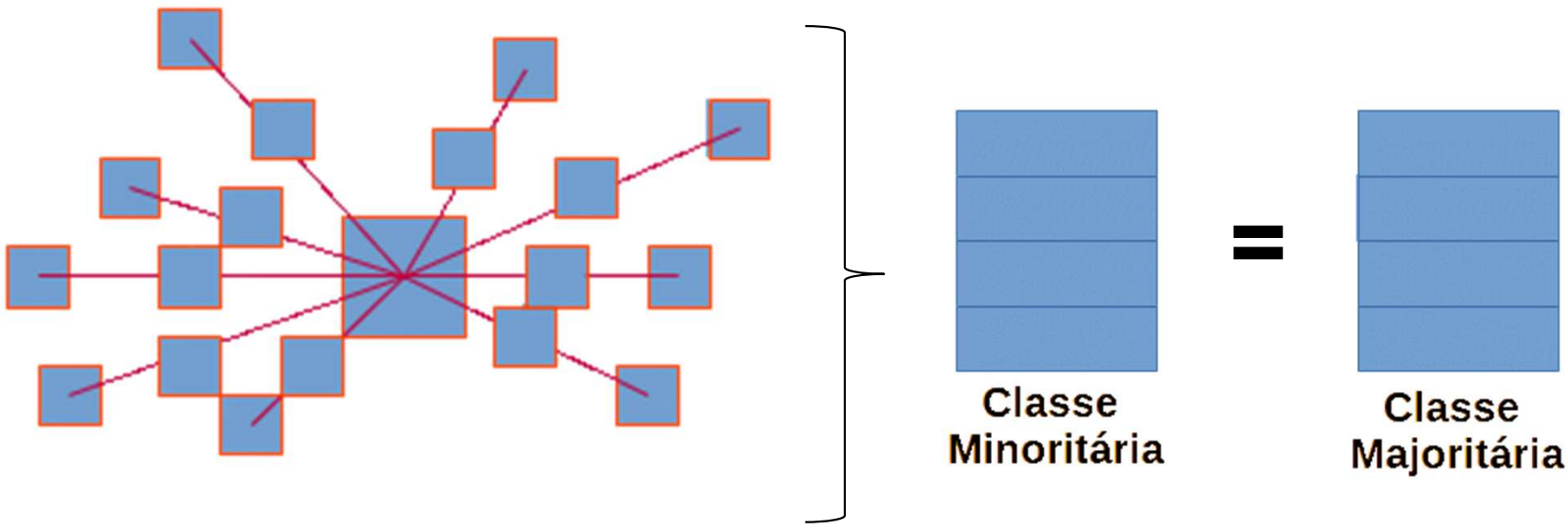


Etapa 5: Para cada dado x_i da Classe Minoritária será gerado g_i dados sintéticos de acordo com os seguintes passos:

INICIA O LOOP de 1 até g_i:

- Escolher aleatoriamente 1 dado da Classe Minoritária x_{zi} dos K-NN do dado x_i;
- Gerar o dado sintético: $s_i = x_i + (x_{zi} - x_i) \times \lambda$
  - onde $(x_{zi} - x_i)$ é o vetor diferença no espaço n dimensional
  - e λ é um número aleatório ($\lambda \in [0,1]$).

FIM do LOOP



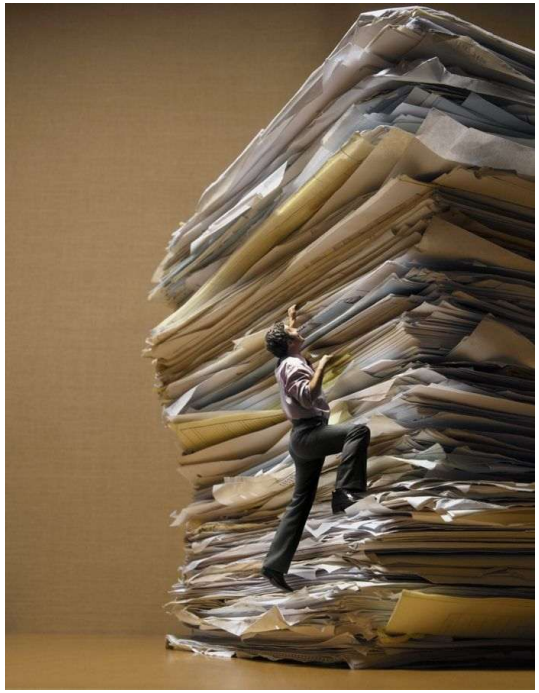
Sugestões de Como Lidar com Desbalanço

- **ESSENCIAL:**
  - **1º Compreender o problema**
  - **2° Priorizar a calibração**
  - **3º Métricas além da AUC ROC**
  - **4° Técnicas de reamostragem**
  - **5° Aprendizado sensível a custos**
  - **6º Ajuste de limiar**

Trabalho Individual

Trabalho Fechamento Área 01 da Disciplina

- Selecionar uma área de aplicação com acesso a um ou mais conjuntos de dados (ideal);
- Realizar uma pesquisa bibliométrica relacionada a área de aplicação selecionada e modelos de *machine learning* utilizados, assim como, os principais autores citados;
- Procurar alguma base de dados de interesse e desenvolver uma EDA manual;
- Pesquisar sobre métodos para EDA automático (como apoio consultar a Nota de Aula 04C e o correspondente laboratório – atividades extraclasse) e utilizar alguma biblioteca automática na base de dados de interesse;
- Discutir seus resultados;
- Apresentar este trabalho no formato de um artigo científico – selecionar alguma revista da área de *machine learning* e utilizar o correspondente *template*;
- trabalho pode ser individual ou em dupla;
- entregar até o dia **14/05/2026**.
- obs.:
  - para auxiliar no trabalho com relação a pesquisa bibliométrica consultar material disponibilizado no Moodle da disciplina.



Atividades de Laboratório

LAB04B

- Acessar o Moodle:
- Executar as tarefas solicitadas
- Vamos discutir as partes essenciais de cada tarefa
- Mãos à obra